

Part 3, Task 4 — Advanced AI Technique: MLflow Experiment Tracking

Why this technique was selected

Across Tasks 1 and 3, six different models were trained (2 classification, 2 regression, 1 LSTM, 1 lag-feature Random Forest for explainability) to solve two different prediction problems. Without a systematic way to track them, comparing results meant re-reading scattered print statements and manually re-running scripts. **MLflow** was chosen because it directly solves this problem, and because it connects naturally to Task 6's MLOps requirements (model versioning, experiment tracking are literally the same MLflow store), using it here means Task 4 and Task 6 reinforce each other rather than duplicating work with two different tools.

How it was implemented

- A single local tracking store is used for the whole project: `part3_machine_learning/mlflow/mlflow.db` (SQLite backend, MLflow's file-store backend is now in maintenance mode, so SQLite was used instead).
- Two experiments are logged: `traffic_supervised_ml` (Task 1's 4 models) and `traffic_deep_learning` (Task 3's LSTM and comparable Random Forest).
- Every model gets its own MLflow **run**, logging:
- **Parameters**, model type, task, and key hyperparameters (e.g. `n_estimators`, `max_depth`, `lookback`, `epochs`).
- **Metrics** — the full evaluation metrics from Tasks 1/3 (accuracy, precision, recall, F1, ROC-AUC for classification; MAE, R^2 for regression; plus training/validation loss for the LSTM).
- **Artifacts** — the LSTM's saved Keras model and the lag-feature Random Forest's joblib file are logged as MLflow artifacts, alongside also being saved directly to `models/` for the deployment/recommendation scripts to load without needing to query MLflow.
- Runs can be browsed with `mlflow ui --backend-store-uri sqlite:///part3_machine_learning/mlflow/mlflow.db`, or queried programmatically with `mlflow.search_runs()` (used to produce the model versioning comparison table in `mlops/model_versioning.md`).

What value it adds

- **Reproducible comparison**: every model's hyperparameters and metrics are in one queryable place instead of scattered across script output, making it trivial to answer "which model actually performed best, and with what settings" months later.
- **Foundation for MLOps** (Task 6): model versioning and experiment tracking are the same underlying store, no separate system needed.
- **Audit trail**: if a deployed model's behaviour is questioned later, the exact training run, parameters and metrics that produced it are retrievable rather than lost.

Limitations

- This is a **local, single-user** tracking setup (SQLite file), not a shared team server, in a real city-government deployment, this would need to move to a hosted MLflow tracking server with proper access control, since a SQLite file both isn't safely shared across multiple users writing concurrently and isn't backed up or access-controlled.

- MLflow tracks *what was trained and how well it did*, but does not by itself handle **data versioning** (which snapshot of `featured_traffic_data.csv` produced a given run) or automatic retraining triggers, those would need to be added separately (e.g. DVC for data versioning, a scheduler for retraining) for a genuinely production-grade MLOps setup.
- Logging was added after model training code was already written, which is workable for a capstone project but a real deployment should design logging in from the start so no run is ever accidentally left untracked.